Rapport d'audit de sécurité

App_muscu (iOS SwiftUI + Supabase)

Audit en boîte blanche – 2026-10-06

Référentiels : OWASP API Security Top 10 2023, OWASP Top 10 for LLM Applications 2025, OWASP MASVS v2, ASVS 5.0.0, CVSS v3.1

Document confidentiel.

Table des matières

1 Résumé exécutif **2**

2 Périmètre, méthode et limites **2**

2.1 Périmètre 2

2.2 Méthode 2

2.3 Limites : ce qui n'a PAS été testé 3

3 Échelle de gravité **4**

4 Synthèse des constats **4**

5 Détail des constats **5**

6 Remarques informatives **9**

7 Contrôles vérifiés sans anomalie **10**

8 Matrice de couverture **10**

8.1 Par composant 11

8.2 OWASP API Security Top 10 2023 11

8.3 OWASP Top 10 for LLM Applications 2025 12

8.4 OWASP MASVS v2 (revue de code seulement) 12

9 Plan de remédiation priorisé **12**

10 Conclusion **13**

1 Résumé exécutif

L'application a été auditée en deux temps : une revue complète du code source (base de données, fonctions serveur, application iOS) puis des attaques réelles contre une copie locale du serveur, avec 9 appels réels à l'IA Claude. Le verdict reste globalement bon : **aucune faille critique ni élevée** n'a été trouvée. Les données d'un utilisateur sont bien isolées de celles des autres (vérifié sur les huit tables), les accès sans authentification ou avec un jeton expiré, falsifié ou mal formé sont refusés, les limites d'usage de l'IA tiennent jusqu'au refus, et toutes les tentatives d'injection de consignes dans l'IA ont échoué.

Deux constats de gravité **moyenne** subsistent : la suppression définitive de compte n'exige qu'un jeton de session valide, sans nouvelle saisie du mot de passe ; et des contraintes de base de données incomplètes permettent, en écrivant directement dans l'API, de stocker des valeurs que les fonctions serveur refuseraient, avec un impact limité aux propres données de l'utilisateur. Les neuf autres constats sont faibles (hygiène de secrets, dépendances non verrouillées, durcissement iOS, limitation des inscriptions à vérifier en production).

L'application n'a pas été corrigée depuis l'audit précédent (v3 du 2026-10-05) : tous les constats sont donc *ouverts*. Plusieurs points ne peuvent pas être jugés depuis un poste de développement : la configuration d'authentification du serveur réellement déployé (limitation des inscriptions, confirmation d'e-mail, durée des sessions) et la solidité de l'application iOS une fois installée (aucune analyse du binaire). Ces points sont listés dans les limites et doivent être vérifiés avant une mise en production.

2 Périmètre, méthode et limites

2.1 Périmètre

— Base Postgres Supabase : 7 migrations (8 tables, politiques RLS, fonctions `consume_ai_quota` et `valid_template_items`).

— 4 Edge Functions Deno/TypeScript : `coach-command`, `coach-review`, `coach-workout`, `delete-account`, et les modules partagés `_shared` (`auth`, `validate`, `review`, `claude`, `build_catalog`).

— Application iOS : tous les fichiers Swift de `ios/Coach` et `ios/CoachWidgets`, `Info.plist` et `xcconfig`.

— Appels à l'API Claude (uniquement depuis les Edge Functions).

Le code de l'application n'a pas été modifié.

2.2 Méthode

Référentiel : avant l'audit, le référentiel `.claude/audit/referentiel_owasp.md` a été revérifié aux sources officielles OWASP (compte rendu du sous-agent dédié, 2026-10-06) : OWASP API Security Top 10 *à jour* (édition 2023), OWASP Top 10 for LLM Applications *à jour* (2025), MASVS *à jour* (8 groupes, v2), ASVS *à jour* (5.0.0, chapitres non relus au dépôt cette fois). Aucune catégorie n'a changé ; l'audit s'appuie donc sur le fichier local confirmé.

Analyse statique : lecture intégrale de chaque migration, fonction et module partagé, puis de tous les fichiers Swift de `ios/Coach` et `ios/CoachWidgets`. Outils réellement lancés : `semgrep -config auto` (1 résultat, faux positif ReDoS sur une regex construite à partir d'une constante, cf. I-07) et `gitleaks detect -no-git` (5343 résultats bruts, tous dans `ios/`

`build/SourcePackages/checkouts` – fixtures de dépendances tierces hors périmètre ; la partie applicative ne révèle que les fichiers de secrets locaux, cf. F-09 et I-01 ; aucun secret suivi par git, vérifié par `git ls-files`).

Test dynamique : contre la stack Supabase *locale* uniquement (`http://127.0.0.1:54321`, aucun hôte `*.supabase.co`), après vérification que GoTrue, PostgREST et `functions serve` répondaient. Tests menés avec la clé *publishable* et des comptes de test (jamais la clé secrète pour juger la RLS ; la clé secrète n'a servi qu'au nettoyage final des comptes). Tous les comptes de test ont été supprimés. Budget d'appels Claude réels : **9 consommés sur 15** (les tests de quota ont été menés via le RPC de quota, sans appel Claude, d'où l'économie).

Notation : chaque constat est classé dans les référentiels OWASP avec un vecteur et un score CVSS 3.1 recalculé par l'orchestrateur avec `cvss31.py`, jamais estimé à la main. Pour les risques sans score CVSS pertinent (abus de coût, hygiène de secrets locaux), la mention ń CVSS non pertinent ż est justifiée dans la fiche.

Vérification des sous-agents et corrections apportées : les constats bruts du scanner (statique) et du pentester (dynamique) ont été recoupés, dédupliqués et revérifiés. Trois corrections ont été appliquées :

— Le scanner avait coté la clé Anthropic en clair sur le poste à **7.5 (Élevée)** avec un vecteur `AV:N`. Un fichier local, ignoré par git et absent du dépôt (confirmé par `git ls-files` : aucun `.env` suivi) n'est pas joignable par le réseau : le vecteur `AV:N` n'est pas justifié. Reclassé en **Faible**, hygiène de secrets locaux, CVSS non pertinent (F-09).

— Le scanner *et* le pentester décrivaient les dépendances `npm:` comme ń épinglées ż. Vérification des specifiers réels : `@supabase/supabase-js@2` et `zod@4` ne fixent que la version majeure (le mineur et le correctif flottent), `@anthropic-ai/sdk@0.128` laisse flotter le correctif, et il n'existe ni `deno.lock` ni `deno.json`. Corrigé en ń dépendances flottantes, pas de lockfile ż (F-06).

— Le pentester avait coté l'ouverture des inscriptions à **5.3 (Moyenne)** à partir du comportement *local* (auto-confirmation par défaut, `AC:L`). Ce comportement est le défaut de développement et ne prouve rien sur la production ; conformément au référentiel (config Auth non vérifiable en local), le vecteur retient `AC:H` et le score est **3.7 (Faible)** tant que la configuration déployée n'est pas vérifiée (F-03).

2.3 Limites : ce qui n'a PAS été testé

— **Configuration Auth du serveur déployé** : il n'existe pas de `supabase/config.toml`. Limitation des inscriptions, confirmation d'e-mail, durée des JWT et politique de mot de passe serveur ne sont pas vérifiables ; seul le comportement par défaut de la stack locale a été observé (cf. F-03, I-08).

— **Application iOS en exécution** : ni simulateur, ni analyse du binaire (MASVS-RESILIENCE non testé, MASVS-CRYPTO sans usage direct de cryptographie dans le code). Le stockage du jeton en Keychain est établi par lecture du code du SDK, pas par inspection d'un appareil.

— **Environnement de production** : aucun test contre un projet Supabase distant.

— **Chaîne d'approvisionnement (LLM03)** : revue des specifiers de dépendances par lecture du code, sans exécution dynamique ni audit amont des paquets publiés.

— Les tests d'injection de prompt (6 variantes sur 9 appels) ne prouvent pas l'absence de toute injection possible : le comportement du modèle n'est pas déterministe.

3 Échelle de gravité

La gravité suit la tranche du score CVSS 3.1 de base.

Niveau	Score	Signification
Critique	9.0–10.0	Exploitation simple, impact majeur sur toutes les données ou le service.
Élevée	7.0–8.9	Compromission importante de données ou de comptes.
Moyenne	4.0–6.9	Impact réel mais conditionné, ou limité en portée.
Faible	0.1–3.9	Durcissement ; exploitation difficile ou impact minime.
Info	0.0	Remarque, contrôle efficace, faux positif ou point à vérifier.

Lorsque aucun score CVSS n'a de sens (abus de coût, hygiène de secrets locaux), la gravité est indiquée avec sa justification. Un score ń mécanique ż plus élevé que la gravité retenue est signalé et justifié par le vecteur dans la fiche.

4 Synthèse des constats

Gravité	Nombre	Identifiants
Critique	0	–
Élevée	0	–
Moyenne	2	F-01, F-02
Faible	9	F-03 à F-11
Info	8	I-01 à I-08

Fiabilité : **S+D** = vu par l'analyse statique et confirmé par le test dynamique (plus fiable) ; **S** = statique seul ; **D** = dynamique seul.

Id	Gravité	Titre	Source	CVSS
F-01	**Moyenne**	Suppression de compte sans réauthentification	S+D	6.8
F-02	**Moyenne**	Contraintes de base de données incomplètes (écriture directe)	S+D	4.3
F-03	**Faible**	Inscriptions non bornées, confirmation d'e-mail non exigée (local)	D, S (non vérifiable)	3.7
F-04	**Faible**	RPC `valid_template_items` exécutable par `anon`	D	3.7
F-05	**Faible**	Messages d'erreur PostgREST verbeux pour l'anonyme	D	5.3 (méca.)
F-06	**Faible**	Dépendances `npm:` flottantes, pas de lockfile	S+D	n/a
F-07	**Faible**	Nom d'exercice du modèle non filtré des caractères de contrôle	S	4.3 (méca.)
F-08	**Faible**	Données locales (santé) en protection ń jusqu'au 1er déverrouillage ż	S	2.0
F-09	**Faible**	Clé Anthropic réelle en clair sur le poste (hors dépôt)	S	n/a
F-10	**Faible**	Données de santé et texte libre envoyés au modèle	S	n/a
F-11	**Faible**	Coût IA : quota par compte, pas de plafond global	S+D	n/a
I-01 à I-08	**Info**	Voir section dédiée	–	–

Statut de tous les constats : *ouvert* (l'application n'a pas été corrigée depuis la v3). ń méca. ż = score CVSS mécanique du vecteur, gravité retenue plus basse et justifiée dans la fiche.

5 Détail des constats

F-01 – Suppression de compte sans réauthentification Moyenne

Classification : OWASP API6 :2023 Unrestricted Access to Sensitive Business Flows ; ASVS gestion de session / contrôle d'accès ; CVSS 3.1 6.8 (AV :N/AC :H/PR :L/UI :N/S :U/C :N/I :H/A :H)
Localisation : `supabase/functions/delete-account/index.ts`
Description : La fonction supprime définitivement le compte dès qu'elle reçoit un jeton d'accès valide et le corps `\{"confirm":"DELETE"\}`. Cette confirmation est une constante connue de n'importe quel client ; aucun mot de passe ni réauthentification récente n'est demandé avant une action irréversible (suppression en cascade de toutes les données de l'utilisateur).
Impact : Toute personne disposant d'un jeton d'accès volé (appareil compromis, cf. F-08) peut détruire définitivement les données du compte (intégrité et disponibilité). L'exploitation suppose un jeton déjà valide, d'où `AC:H`.
Preuve : Dynamique : le happy-path a été prouvé – un compte de test a été supprimé avec son seul jeton et `\{"confirm":"DELETE"\}` (réponse `\{"deleted":true\}`, cascade effective). Les deux agents ont par ailleurs confirmé que le contrôle d'accès est sain : aucun ciblage d'autrui (pas de paramètre d'identité ; une clé `user_id`/`target_user` en trop renvoie 400 ; le handler n'agit que sur `caller.user.id`) et la session est révoquée (`signOut global`). Le constat porte uniquement sur l'absence de réauthentification.
Recommandation : Exiger une réauthentification récente (mot de passe ou jeton à usage unique) avant la suppression définitive.
Statut : Ouvert. Fiabilité S+D.

F-02 – Contraintes de base de données incomplètes (écriture directe) Moyenne

Classification : OWASP API3 :2023 Broken Object Property Level Authorization ; spécifique Supabase (contraintes base) ; ASVS validation des entrées ; CVSS 3.1 4.3 (AV :N/AC :L/PR :L/UI :N/S :U/C :N/I :L/A :N)
Localisation : `supabase/migrations/0007_exercise_key_format.sql:5,8`, `0006_templates_warmup.sql:6-27` (`valid_template_items`)
Description : Trois contraintes de format sont trop faibles. (1) La contrainte 0007 sur `workout_sets`/`performance_goals` utilise le motif `custom:[0-9a-fA-F-]\{36\}` qui accepte 36 caractères hex ou tirets quelconques (pas un vrai UUID) et toute chaîne `[a-z0-9_]+` hors catalogue. (2) Elle est posée `NOT VALID` : les lignes existantes ne sont pas re-contrôlées (impact nul à ce jour, ń no rows exist yet ż). (3) La clé d'exercice des éléments d'un modèle de séance (`session_templates.items`) n'est validée qu'en longueur et en type par `valid_template_items`, sans contrainte de catalogue ni de jeu de caractères – plus faible que la contrainte des autres tables. Un client modifié écrivant directement via PostgREST peut donc stocker des valeurs hors format.
Impact : Données incohérentes ou hostiles dans ses propres lignes. Aucun chemin vers d'autres utilisateurs : la RLS isole les lignes, les valeurs sont liées en paramètres (pas d'injection SQL) et ces items de modèle ne sont pas réinjectés dans les prompts Claude des fonctions auditées. Le score Moyen vient du vecteur (`AC:L`) ; l'impact pratique reste faible.
Preuve : Dynamique : `POST /rest/v1/workout_sets` avec `exercise_key:"custom:----------"` (36 tirets) renvoie

201 (accepté) ; `POST /rest/v1/session_templates` avec un item `exercise_key:"totally_made_up_key_<script>"` renvoie 201 (stocké tel quel). À l'inverse, la contrainte 0007 rejette bien un format invalide à l'insert (`"BAD KEY; DROP TABLE x"` → 400), donc `NOT VALID` n'affaiblit pas les écritures nouvelles.
Recommandation : Ajouter un `CHECK` de format réel (UUID) sur les clés ń custom ż, aligner la validation de `valid_template_items` sur celle des autres tables (catalogue + jeu de caractères), et `VALIDATE CONSTRAINT` les contraintes `NOT VALID`.
Statut : Ouvert. Fiabilité S+D.

F-03 – Inscriptions non bornées, confirmation d'e-mail non exigée (stack locale) Faible

Classification : OWASP API2 :2023 Broken Authentication, API6 :2023, API4 :2023 ; ASVS authentification ; CVSS 3.1 3.7 (AV :N/AC :H/PR :N/UI :N/S :U/C :N/I :N/A :L)
Localisation : Configuration Auth (absence de `supabase/config.toml`) ; `ios/Coach/Features/Auth/AuthView.swift`
Description : Sur la stack locale, les inscriptions réussissent sans limitation et un compte reçoit un jeton immédiatement utilisable, `email_confirmed_at` étant renseigné sans vérification. L'absence de `config.toml` empêche de savoir ce qui est réellement configuré sur le projet déployé. La longueur minimale du mot de passe (8) n'est contrôlée que dans l'interface, et il n'y a pas de MFA.
Impact : Création de comptes en série, qui multiplie les quotas IA par compte (cf. F-11) et peut dégrader le service. Le vecteur retient `AC:H` car la faisabilité dépend d'une configuration déployée inconnue ; la gravité reste Faible tant que ce n'est pas vérifié en production.
Preuve : Dynamique : inscriptions successives acceptées, jeton obtenu avec e-mail auto-confirmé. Ces comportements sont les valeurs par défaut du développement local et ne prouvent rien sur la production – le pentester les avait cotés 5.3 (`AC:L`), ramené à 3.7 (`AC:H`) par l'orchestrateur pour cette raison.
Recommandation : Versionner une configuration Auth (confirmation d'e-mail, limites de débit, politique de mot de passe, durée des JWT) et la vérifier sur le projet réel ; envisager un captcha.
Statut : Ouvert (à vérifier en production). Fiabilité D, S non vérifiable.

F-04 – RPC `valid_template_items` exécutable par `anon` Faible

Classification : OWASP API9 :2023 Improper Inventory Management, API5 :2023 ; spécifique Supabase (revoke) ; CVSS 3.1 3.7 (AV :N/AC :H/PR :N/UI :N/S :U/C :N/I :N/A :L)
Localisation : `supabase/migrations/0006_templates_warmup.sql:6-27`
Description : Cette fonction pure (utilisée dans un `CHECK`) n'a pas de `revoke execute` pour `anon`/`public` et apparaît dans l'API `/rpc` sans authentification. Aucun effet de bord, aucun accès aux tables. À comparer avec `consume_ai_quota`, correctement révoquée (appel anonyme refusé en 401).
Impact : Appel de calcul `jsonb` arbitraire sans compte (consommation de ressources mineure). Impact quasi nul ; peut être jugé volontaire.
Preuve : Dynamique : `POST /rest/v1/rpc/valid_template_items` sans JWT renvoie 200 (`true`/`false`) ; la fonction figure dans l'OpenAPI anonyme. Le pentester l'a d'abord classée ń info ż (impact nul) ; retenue en Faible pour l'hygiène d'exposition.
Recommandation : `revoke execute on function ... from anon, public` (le `CHECK` continue de fonctionner).
Statut : Ouvert. Fiabilité D.

F-05 – Messages d'erreur PostgREST verbeux pour l'anonyme Faible

Classification : OWASP API8 :2023 Security Misconfiguration ; CVSS 3.1 vecteur 5.3 (AV :N/AC :L/PR :N/UI :N/S :U/C :L/I :N/A :N), gravité retenue Faible (voir fiche)
Localisation : API REST PostgREST (`GET /rest/v1/<table>` sans JWT)
Description : Un appel anonyme (clé publishable seule) sur une table protégée renvoie un message d'erreur qui cite le nom de la table et la syntaxe exacte du `GRANT` à accorder au rôle `anon`. C'est le comportement par défaut de PostgREST.
Impact : Le vecteur mécanique est 5.3 (divulgation `C:L`). La gravité est ramenée à **Faible** car l'information divulguée (noms de tables, rôles internes) est déjà exposée à l'anonyme par le schéma OpenAPI de PostgREST : la divulgation marginale est négligeable (confidentialité effective proche de nulle). Aucune donnée métier n'est révélée.
Preuve : Dynamique : `GET /rest/v1/profiles` anonyme → 401 avec `"hint":"Grant the required privileges to the current role with: GRANT SELECT ON public.profiles TO anon;"` ; même motif sur les 8 tables.
Recommandation : Désactiver les hints verbeux de PostgREST en production (niveau de détail des erreurs) si l'exposition n'est pas souhaitée.
Statut : Ouvert. Fiabilité D.

F-06 – Dépendances `npm:` flottantes, pas de lockfile Faible

Classification : OWASP LLM03 :2025 Supply Chain ; CVSS non pertinent (aucune exploitation démontrée)
Localisation : `_shared/auth.ts:3-4`, `_shared/claude.ts:2-4`, `delete-account/index.ts:3-4`
Description : `@supabase/supabase-js@2` et `zod@4` ne fixent que la version majeure (mineur et correctif flottent) ; `@anthropic-ai/sdk@0.128` fixe le mineur mais laisse flotter le correctif. Il n'existe ni `deno.lock` ni `deno.json` dans le projet.
Impact : Build non reproductible : une version publiée après coup peut être embarquée sans revue. Risque latent, non exploité.
Preuve : Statique : lecture des specifiers (vérifiée par l'orchestrateur). Le scanner et le pentester les avaient décrits comme ń épinglés ż : correction apportée, `@2` et `@4` sont des plages de version majeure, pas des épinglages, et aucun lockfile n'existe.
Recommandation : Épingler des versions complètes et committer un `deno.lock`.
Statut : Ouvert. Fiabilité S+D.

F-07 – Nom d'exercice produit par le modèle non filtré des caractères de contrôle Faible

Classification : OWASP LLM05 :2025 Improper Output Handling ; ASVS validation des entrées ; CVSS 3.1 vecteur 4.3 (AV :N/AC :L/PR :L/UI :N/S :U/C :N/I :L/A :N), gravité retenue Faible (voir fiche)
Localisation : `supabase/functions/coach-command/index.ts:46`, `_shared/validate.ts:148`
Description : `CommandOut.custom_name` et `custom_muscle_group` (noms proposés par le modèle) sont validés contre les URL et une liste de mots interdits, mais pas contre les caractères de contrôle `\textbackslash p\{Cc\}` – contrairement à `freeName` de `review.ts`. Un nom peut donc contenir des retours ligne ou caractères de contrôle, être stocké, puis réaffiché.
Impact : Incohérence de validation. Impact concret faible : la sortie est rendue dans un `Text` SwiftUI (aucun interpréteur HTML/shell), d'où la gravité Faible malgré le vecteur mécanique 4.3.
Preuve : Statique : lecture du code ; `checkCommand` ne nettoie pas `\textbackslash`

`p\{Cc\}` alors que `review.ts` le fait pour les noms libres.
Recommandation : Appliquer le même filtre `\textbackslash p\{Cc\}` aux noms générés avant stockage.
Statut : Ouvert. Fiabilité S.

F-08 – Données locales (santé) en protection ń jusqu'au premier déverrouillage ż Faible

Classification : OWASP MASVS-STORAGE, MASVS-PRIVACY ; CVSS 3.1 2.0 (AV :P/AC :H/PR :N/UI :N/S :U/C :L/I :N/A :N)
Localisation : `ios/Coach/Models/Store.swift:5-14`
Description : La base SwiftData (profil, séances, poids de corps, zones de blessure) est conservée en protection ń jusqu'au premier déverrouillage ż (choix assumé pour permettre l'écriture par les intents de la Live Activity depuis l'écran verrouillé). Les jetons de session, eux, sont au Keychain (SDK supabase-swift), non concernés.
Impact : Lecture des données de santé sur un appareil physiquement compromis et déjà déverrouillé une fois (ou jailbreaké). Un jeton volé alimenterait F-01.
Preuve : Statique : lecture du code et du SDK ; pas d'inspection d'un appareil.
Recommandation : Utiliser la protection `complete` pour la base si cela reste compatible avec les écritures hors écran verrouillé.
Statut : Ouvert. Fiabilité S.

F-09 – Clé Anthropic réelle en clair sur le poste (hors dépôt) Faible

Classification : OWASP ASVS gestion des secrets ; API8 ; spécifique Supabase (clés hors du poste) ; CVSS non pertinent (hygiène de secrets locaux)
Localisation : `supabase/functions/.env`
Description : Ce fichier contient une clé `ANTHROPIC_API_KEY` réelle en clair. Il est ignoré par git (`.env*`) et absent du dépôt (aucun `.env` suivi, vérifié par `git ls-files`).
Impact : Une clé Anthropic réelle en clair sur disque est exposée en cas de compromission du poste ou de sauvegarde non chiffrée. Le scanner l'avait cotée 7.5 (Élevée) avec `AV:N` ; un fichier local hors dépôt n'étant pas joignable par le réseau, le vecteur `AV:N` n'est pas justifié et la gravité est ramenée à Faible (hygiène de secrets locaux, CVSS non pertinent).
Preuve : Statique : gitleaks et lecture directe ; valeur non reproduite ici.
Recommandation : Stocker la clé dans un gestionnaire de secrets, la régénérer si elle a été partagée, restreindre son usage.
Statut : Ouvert. Fiabilité S.

F-10 – Données de santé et texte libre envoyés au modèle Faible

Classification : OWASP LLM02 :2025 Sensitive Information Disclosure, MASVS-PRIVACY ; CVSS non pertinent (minimisation de données)
Localisation : `supabase/functions/coach-command/index.ts:30`, `coach-workout/index.ts:51`
Description : Les zones de blessure/douleur (`avoid_zones`) et des noms d'exercices libres sont transmis à l'API Claude. Ni l'e-mail ni l'identifiant ne sont envoyés, aucune donnée d'un autre utilisateur n'entre dans le contexte, et le poids de corps n'est pas transmis (seuls des agrégats). Le contenu n'est pas journalisé (`claude.ts` ne logge que la durée).
Impact : Une donnée de santé saisie par l'utilisateur est transmise au fournisseur tiers. Minimisation globalement correcte ; signalé pour traçabilité PII.
Preuve : Statique : lecture du code ; les tentatives dynamiques d'obtenir des données d'autrui ont échoué.
Recommandation : Informer l'utilisateur dans la politique de confidentialité ; envisager

un avertissement dans le champ libre.
Statut : Ouvert. Fiabilité S.

F-11 – Coût IA : quota par compte, pas de plafond global **Faible**

Classification : OWASP API4 :2023 Unrestricted Resource Consumption, LLM10 :2025 Unbounded Consumption ; CVSS non pertinent (abus de coût)
Localisation : `_shared/claude.ts`, `_shared/auth.ts`, migration 0002
Description : Les quotas sont de 5 (workout), 20 (parse/command) et 3 (review) appels par jour et par compte, avec un `max_tokens` borné, mais il n'y a aucun plafond global ni limite par IP. Le coût croît avec le nombre de comptes (cf. F-03).
Impact : Facture Anthropic non bornée en cas d'inscriptions automatisées. Gravité Faible car borné par compte et dépendant d'une configuration non vérifiée.
Preuve : Dynamique : quotas workout (429 au 6e appel), parse (429 au 21e) et review (429 au 4e) atteints ; refus *avant* tout appel Claude. La limite est codée en dur dans la fonction `security definer`, inchangeable par l'appelant ; l'insertion directe dans `ai_usage` est refusée (403).
Recommandation : Ajouter un plafond global journalier (compteur ou limite côté fournisseur), des alertes de coût, et limiter les inscriptions (F-03).
Statut : Ouvert. Fiabilité S+D.

6 Remarques informatives

I-01 **Secrets par défaut de la stack locale** (API2/API8, CVSS non pertinent) : `.env.test` et `supabase/.temp/start-secrets/.../docker.env` contiennent la clé `service_role` et le secret JWT par défaut de `supabase start` (valeurs publiques, `iss=supabase-demo`), ignorés par git et limités à `127.0.0.1`. Sans portée production, mais devient critique si ce secret était réutilisé en production.

I-02 **RLS non ń forcée ż** (spécifique Supabase) : aucune table n'a `FORCE ROW LEVEL SECURITY`. Sans effet pratique pour PostgREST (le rôle `authenticated` n'est pas propriétaire des tables) ; durcissement de défense en profondeur.

I-03 **CORS** (API8) : `Access-Control-Allow-Origin: *` sur les Edge Functions (défaut de la passerelle Kong locale). Sans conséquence pour une API à jeton Bearer sans cookie.

I-04 **Jeton d'accès valide jusqu'à expiration après suppression de compte** (API2) : après `delete-account`, l'access-token reste accepté par PostgREST jusqu'à son `exp` (limite JWT standard), mais la cascade a supprimé toutes les lignes (retour `[]`) et les Edge Functions le refusent (`getUser` échoue → 401) ; les refresh tokens sont révoqués. Impact nul.

I-05 **Clé publishable embarquée côté iOS** (`ios/Config.xcconfig`) : publique par conception ; aucune clé secrète ni `service_role` côté client.

I-06 **Garde-fou ń pas de conseil médical ż uniquement dans le prompt** (LLM09) : la sortie n'est pas filtrée pour des termes médicaux ; le code ne contrôle que URL, mots interdits et nombres. La garde anti-invention de nombres (`checkReview` : tout nombre doit provenir des faits fournis) a été confirmée dynamiquement. Durcissement possible : contrôle de sortie et avertissement affiché.

I-07 **Faux positif semgrep** `detect-non-literal-regexp` (`_shared/review.ts:56`) : l'expression est construite à partir d'une constante (`FORBIDDEN_WORDS`), jamais d'une entrée utilisateur.

I-08 **Configuration Auth déployée non vérifiable** : pas de `supabase/config.toml` ; confirmation d'e-mail, limites de débit, durée des JWT et maintien de `verify_jwt` à vérifier sur le projet réel (cf. F-03).

7 Contrôles vérifiés sans anomalie

— **RLS** activée sur les 8 tables, politiques filtrées par `(select auth.uid())`, aucun `using (true)`, droits `anon` révoqués. Isolation A/B vérifiée dynamiquement en lecture, modification (PATCH no-op), insertion avec usurpation de `user_id` (403) et suppression croisée (no-op), sur les 8 tables nommément.

— **Clés étrangères croisées** : insertion de `workout_sets` référençant le `workout_id` d'un autre utilisateur refusée (403).

— **Authentification des fonctions** : sans jeton, jeton malformé, jeton expiré, jeton forgé signé par une autre clé, mauvaise méthode (405) : tous refusés (double barrière passerelle `verify_jwt` + `requireUser` qui relit l'utilisateur via `getUser`).

— `delete-account` : corps strict (`strictObject`), confirmation littérale `DELETE`, pas de ciblage d'autrui, session révoquée après suppression. Seul l'absence de réauthentification est retenue (F-01).

— `consume_ai_quota` : `security definer` avec `search_path` fixé, limites codées en dur, appel anonyme refusé, pas d'injection dans `p_kind` (paramétré).

— **Clé** `service_role` : présente uniquement dans `delete-account`, jamais côté client ni suivie par git.

— **Injection SQL** : aucune concaténation SQL ; payloads SQL envoyés stockés inertes ou rejetés par contrainte.

— **Prompt injection / LLM** : fuite du prompt système, exfiltration de données d'autrui, contournement des règles métier (charges/volumes), injection via `session_in_progress`/`recent`/noms libres : toutes refusées (sorties contraintes par schéma Zod puis revalidées par `checkWorkout`/`checkCommand`/`checkReview`). Le modèle n'a aucun outil ni accès base (API10, LLM05, LLM06, LLM07).

— **iOS** : aucune exception ATS, HTTPS forcé, jetons en Keychain, aucune clé en dur hormis la publishable, réponses affichées en `Text` simple, décodage tolérant côté synchronisation.

— **SSRF** : aucune URL fournie par l'utilisateur n'est consommée côté fonction.

— **Requêtes invalides** (sans Content-Type, JSON invalide, corps vide, clés en trop) : 400 générique sur les 4 fonctions.

8 Matrice de couverture

Fusion des matrices du scanner (S, statique) et du pentester (D, dynamique). Légende : **OK** testé/lu sans anomalie ; **C** constat ; **NA** non applicable ; **NT** non testé / non lu.

8.1 Par composant

Composant	S	D	Remarque
Tables (8) et RLS	OK	OK	Isolation A/B ; anon GET/POST refusés
Contraintes `exercise_key` / `session_templates.items`	C	C	F-02
RPC `consume_ai_quota`	OK	OK	Révoquée pour `anon`
RPC `valid_template_items`	OK	C	F-04
OpenAPI `/rest/v1/`	NA	C	F-05 (hints verbeux)
`coach-command`, `coach-workout`, `coach-review`	OK	OK	Appel réel chacune ; injections résistées
`delete-account`	C	C	F-01 ; contrôle d'accès sain
Modules `_shared`	C	OK	F-06, F-07, F-11
Migrations 0001 à 0007	C	OK	F-02, F-04, I-02
Application iOS (tous fichiers Swift et widgets)	C	NT	F-08, F-10 ; pas d'exécution
Configuration Auth déployée	NT	NT	Pas de `config.toml` ; F-03, I-08
Binaire iOS (RESILIENCE)	NT	NT	Pas d'analyse de binaire
Secrets et configuration	C	OK	F-09, I-01, I-05

8.2 OWASP API Security Top 10 2023

Catégorie	S	D	Constats / raison
API1 Broken Object Level Authorization	OK	OK	Isolation A/B sur 8 tables
API2 Broken Authentication	OK	OK	JWT absent/expiré/forgé/malformé testés ; F-03, I-04
API3 Object Property Level Authorization	C	C	F-02 ; `user_id` usurpé refusé
API4 Unrestricted Resource Consumption	C	OK	F-11 ; quotas épuisés (429)
API5 Function Level Authorization	OK	C	F-04 ; droits `anon` testés
API6 Sensitive Business Flows	C	C	F-01, F-03
API7 SSRF	NA	NA	Aucune URL utilisateur côté fonction
API8 Security Misconfiguration	C	C	F-05, I-01, I-03 ; config Auth distante NT
API9 Improper Inventory Management	OK	C	F-04 ; OpenAPI vérifié
API10 Unsafe Consumption of APIs	OK	OK	Sorties du modèle revalidées côté serveur

8.3 OWASP Top 10 for LLM Applications 2025

Catégorie	S	D	Constats / raison
LLM01 Prompt Injection	OK	OK	6 variantes, aucune exploitation
LLM02 Sensitive Information Disclosure	C	OK	F-10 ; pas de donnée d'autrui
LLM03 Supply Chain	C	NT	F-06 ; audit amont des paquets non fait
LLM04 Data and Model Poisoning	NA	NA	Pas d'entraînement ni d'ajustement
LLM05 Improper Output Handling	C	OK	F-07 ; sorties revalidées, affichage en texte simple
LLM06 Excessive Agency	OK	OK	Le modèle n'a aucun outil ni action
LLM07 System Prompt Leakage	OK	OK	Fuite demandée, non obtenue
LLM08 Vector and Embedding Weaknesses	NA	NA	Pas de base vectorielle ni d'embeddings
LLM09 Misinformation	C	C	I-06 ; anti-invention de nombres confirmée
LLM10 Unbounded Consumption	C	OK	F-11 ; quotas épuisés

8.4 OWASP MASVS v2 (revue de code seulement)

Groupe	Statut	Remarque
STORAGE	Revue de code	F-08
CRYPTO	Revue de code	Aucun usage direct de cryptographie (délégué au SDK/Keychain)
AUTH	Revue de code	Jetons en Keychain (SDK)
NETWORK	Revue de code	ATS sans exception, HTTPS seul
PLATFORM	Revue de code	Intents Live Activity dans le process app
CODE	Revue de code	Décodage tolérant, bornes appliquées côté app
RESILIENCE	NT	Pas d'analyse de binaire ni de simulateur
PRIVACY	Revue de code	F-08, F-10, I-06

9 Plan de remédiation priorisé

1. **Avant toute mise en production** : versionner et vérifier la configuration Auth du projet réel (limitation des inscriptions, confirmation d'e-mail, politique de mot de passe, durée des JWT) ; ne jamais réutiliser le secret JWT local (F-03, I-01, I-08).
2. **Court terme** : exiger une réauthentification pour `delete-account` (F-01) ; compléter les contraintes de base (F-02) et révoquer `valid_template_items` à `anon` (F-04).
3. **Coût et abus** : plafond global d'appels IA et alertes de coût (F-11) ; régénérer et mieux ranger la clé Anthropic (F-09).
4. **Durcissement serveur** : réduire la verbosité des erreurs PostgREST en production (F-05) ; filtrer les caractères de contrôle des noms générés (F-07) ; envisager `FORCE RLS` (I-02).
5. **Chaîne d'approvisionnement** : épingler les versions `npm:` et committer un `deno.lock` (F-06).
6. **iOS, durcissement** : protection `complete` du stockage local (F-08) ; information de confidentialité sur les données envoyées au modèle (F-10).
7. **Vérifications restantes** : analyse du binaire et test sur appareil (MASVS-RESILIENCE) ; test sur un environnement Supabase de préproduction.

10 Conclusion

Le backend reste solide sur les points qui comptent le plus : isolation entre utilisateurs (vérifiée dynamiquement sur les huit tables), authentification des fonctions, quotas IA et résistance aux injections testées. Il subsiste deux constats de gravité moyenne (réauthentification de la suppression de compte, contraintes de base incomplètes) et neuf faibles, tous ouverts, sans faille critique ni élevée. La priorité avant production est la vérification de la configuration d'authentification du serveur déployé, que cet audit n'a pas pu évaluer, puis la réauthentification de la suppression de compte et la complétion des contraintes de base. Aucun secret n'est reproduit dans ce rapport.